Practical Deep Learning, 2nd Edition

A Python-Based Introduction

by Ronald T. Kneusel

Errata updated to print 1

<table>
<tr><th>Page</th><th>Error</th><th>Correction</th><th>Print corrected</th></tr>
<tr><td>31</td><td>

```
>>> x, y = make_classification(n_samples=10000, weights=(0.9, 0.1))
```

</td><td>

```
>>> x, y = make_classification(n_samples=10000, weights=[0.9, 0.1])
```

</td><td>Pending</td></tr>
<tr><td>32</td><td>

```
a, b = make_classification(n_samples=10000, weights=(0.9, 0.1))
```

</td><td>

```
a, b = make_classification(n_samples=10000, weights=[0.9, 0.1])
```

</td><td>Pending</td></tr>
<tr><td>34</td><td>

```
x, y = make_classification(n_samples=10000, weights=(0.9, 0.1))
```

</td><td>

```
x, y = make_classification(n_samples=10000, weights=[0.9, 0.1])
```

</td><td>Pending</td></tr>
<tr><td>273</td><td>

Experiment	Modification	Test accuracy	Parameters
1	Add Conv_3, 3×3×64 before `Pooling`	94.11%	2,076,554

</td><td>

Experiment	Modification	Test accuracy	Parameters
1	Add Conv_3, 3×3×64 before `Pooling`	94.11%	1,048,394

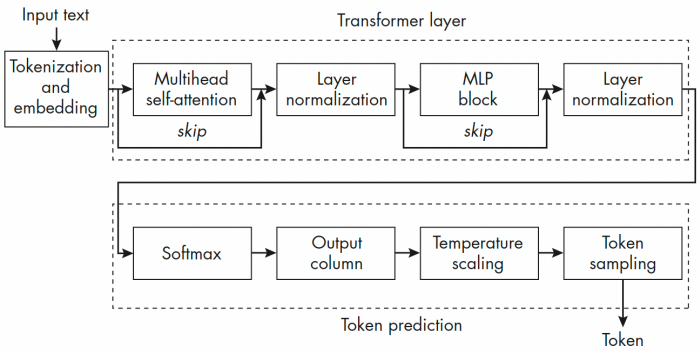
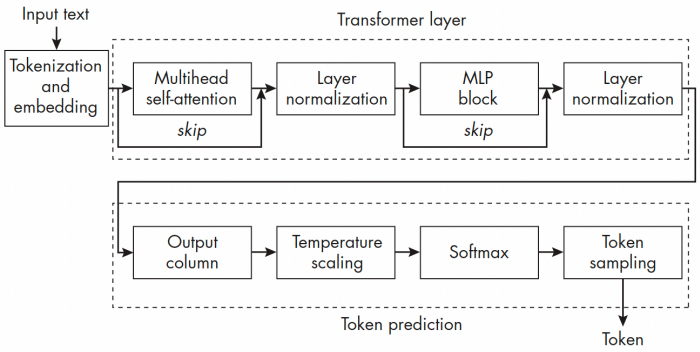
</td><td>Pending</td></tr>
<tr><td>280</td><td>

```
model.add(Conv2D(32, kernel_size=(3, 3),
                 activation='relu',
❷ input_shape=(None, None, -1)))
```

</td><td>

```
model.add(Conv2D(32, kernel_size=(3, 3),
                 activation='relu',
❷ input_shape=(None, None, 1)))
```

</td><td>Pending</td></tr>
</table>

Page	Error	Correction	Print corrected
496	 *Figure 18-1: An LLM block diagram*	 *Figure 18-1: An LLM block diagram*	Pending
502	The next step passes ***p*** through a **softmax to transform the elements into probabilities over the vocabulary. The probabilities are informed by the model's actions via the transformer layer(s). All that remains is to use the probability distribution to predict the emitted token. However, before that, we have a final, short detour to take to understand what is meant by an LLM's temperature, the parameter it uses to influence the likelihood of the next token emitted** .	The next step passes ***p*** through a **function to adjust its temperature, the parameter the model uses to influence the likelihood of the next token emitted** .	Pending
504	To conclude this lengthy section: The **model selects the most likely token if *T* = 0; otherwise, it samples from *p* after adjusting it via a specified temperature, *T*.** The token emitted is added to the prompt, and the process repeats until the EOS token is produced.	To conclude this lengthy section: The **temperature-adjusted output is passed through a softmax to convert it into a probability distribution. Then, the model selects the most likely token.** The token emitted is added to the prompt, and the process repeats until the EOS token is produced.	Pending